Task-16:- Requirement Analysis

# 1. Introduction

### 1.1 Purpose

The purpose of this document is to describe the functional and non-functional requirements for deploying and managing a Flask-based web application on AWS infrastructure. The system will use multiple EC2 instances behind an Application Load Balancer and automate deployments using AWS Systems Manager (SSM) Run Command.

This document is intended for:

- DevOps Engineers
- Cloud Engineers
- System Administrators
- Project Stakeholders
- Developers responsible for maintaining the Flask application

The document provides a detailed understanding of system architecture, requirements, constraints, and operational workflows.

---

### 1.2 Scope

The scope of this project includes designing and deploying a scalable and automated infrastructure for a Flask web application.

#### Included in Scope

- Deployment of Flask application on **two EC2 instances**
- Configuration of **Application Load Balancer (ALB)**
- Load balancing across **multiple Availability Zones**
- Integration with **MongoDB Atlas cloud database**
- Automated deployment using **AWS SSM Run Command**
- Logging deployment results in `/var/log/deploy.log`
- Health checks for application monitoring
- Secure access to EC2 instances through ALB

**Not Included in Scope**

- CI/CD pipeline implementation (GitHub Actions, Jenkins, etc.)
- Database management or backup (handled by MongoDB Atlas)
- Frontend application development
- Monitoring tools such as Prometheus or Grafana

---

### 1.3 Definitions, Acronyms, and Abbreviations

| Term | Description |
|---|---|
| AWS | Amazon Web Services |
| EC2 | Elastic Compute Cloud |
| ALB | Application Load Balancer |
| AZ | Availability Zone |
| SSM | AWS Systems Manager |
| SSM Run Command | Tool used to remotely execute commands on EC2 instances |
| MongoDB Atlas | Cloud-managed MongoDB database service |
| Gunicorn | Python WSGI HTTP server for running Flask applications |
| Deployment | Process of updating the application code on servers |
| Health Check | Mechanism used to verify the availability of application instances |

---

### 1.4 References

The following resources were used while designing this system:

- AWS EC2 Documentation
  https://docs.aws.amazon.com/ec2/

- AWS Systems Manager Documentation
  https://docs.aws.amazon.com/systems-manager/
- AWS Application Load Balancer Documentation
  https://docs.aws.amazon.com/elasticloadbalancing/
- MongoDB Atlas Documentation
  What is MongoDB Atlas? - Atlas - MongoDB Docs
- Flask Framework Documentation
  Welcome to Flask — Flask Documentation (3.1.x)

---

### 1.5 Overview

This document is organized as follows:

- **Section 1** – Introduction to the project
- **Section 2** – Project overview and objectives
- **Section 3** – Functional requirements and system use cases
- **Section 4** – Non-functional system requirements
- **Section 5** – Assumptions and project dependencies
- **Section 6** – Risk analysis and mitigation strategies
- **Section 7** – Glossary of technical terms
- **Section 8** – Appendices
- **Section 9** – Customer acknowledgement

---

## 2. Project Overview

### 2.1 Project Objectives

The main objectives of this project are:

- Deploy a **Flask-based application** across multiple EC2 instances.
- Configure **high availability using AWS Application Load Balancer**.
- Automate deployment using **AWS SSM Run Command**.
- Integrate application with **MongoDB Atlas cloud database**.
- Implement **centralized deployment logging**.
- Ensure **secure access and scalable architecture**.

---

### 2.2 Business Goals

The project aims to achieve the following business goals:

- Increase **application availability and fault tolerance**.
- Improve **deployment efficiency through automation**.
- Reduce **manual server management tasks**.
- Provide **scalable infrastructure for growing traffic**.
- Ensure **secure communication between application components**.

---

### 2.3 Background

Modern cloud applications require scalable and highly available infrastructure. Traditional single-server deployments often lead to downtime and limited scalability.

To overcome these issues, organizations use:

- Load balancing
- Automated deployments
- Managed database services

This project implements these practices using AWS services such as:

- EC2
- Application Load Balancer
- AWS Systems Manager
- MongoDB Atlas

This architecture ensures improved reliability, scalability, and simplified management.

---

### 2.4 Constraints

The following constraints affect the project:

| Constraint | Description |
|---|---|
| Time | Project must be completed within **3 hours** |
| Infrastructure | Limited to AWS services available in the account |

| | |
|---|---|
| Application Type | Only Flask-based backend application |
| Deployment Method | Must use **AWS SSM Run Command** |
| Database | Must use **MongoDB Atlas cloud database** |

---

## 3. Functional Requirements

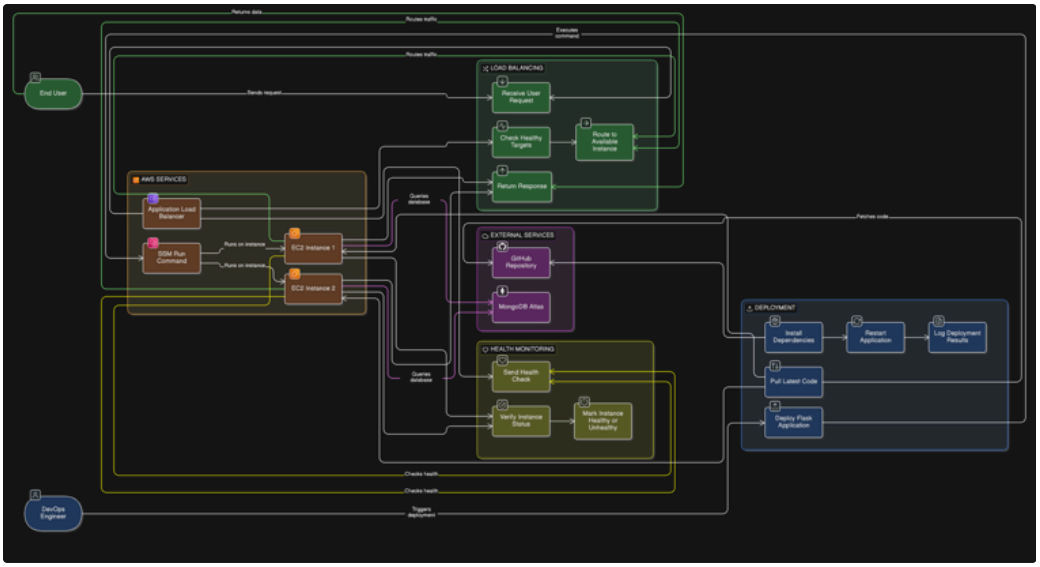
### 3.1 Use Case Diagram

Actors involved in the system:

- DevOps Engineer
- AWS SSM
- Application Load Balancer
- Flask Application Server
- MongoDB Atlas

Suggested diagram flow:

```
DevOps Engineer
      |
      v
AWS SSM Run Command
      |
      v
EC2 Flask Servers (2 Instances)
      |
      v
Application Load Balancer
      |
      v
End Users
      |
      v
MongoDB Atlas Database
```




---

## 3.2 Use Cases

---

### 3.2.1 Use Case 1: Deploy Flask Application

#### Description

Deploy the latest version of the Flask application on EC2 instances using AWS SSM Run Command.

#### Actors

- DevOps Engineer
- AWS SSM

#### Preconditions

- EC2 instances must be running
- SSM agent installed
- EC2 instances attached to SSM IAM role
- Flask repository available on GitHub

#### Postconditions

- Latest code deployed
- Application restarted
- Deployment logs stored in `/var/log/deploy.log`

#### Flow of Events

1. DevOps Engineer executes the **SSM Run Command**.

2. SSM document `DeployFlaskApp` runs on EC2 instances.
3. EC2 instances pull the latest code from GitHub.
4. Dependencies are installed using `pip install -r requirements.txt`.
5. Flask application is restarted using systemd or Gunicorn.
6. Deployment logs are stored in `/var/log/deploy.log`.

#### Alternative Flows

**Alternative Flow 1 – Dependency installation failure**

- Deployment stops.
- Error is logged in `/var/log/deploy.log`.

**Alternative Flow 2 – GitHub repository unavailable**

- Deployment fails.
- System retries deployment after issue resolution.

---

### 3.2.2 Use Case 2: Load Balancing Requests

#### Description

Distribute incoming user requests across multiple Flask servers.

#### Actors

- End Users
- Application Load Balancer

#### Preconditions

- ALB must be configured
- EC2 instances registered in target group
- Health checks must pass

#### Postconditions

- User request processed successfully by one of the EC2 instances.

#### Flow of Events

1. User sends request to **ALB DNS endpoint**.
2. ALB receives the request.
3. ALB checks available healthy targets.

4. Request is forwarded to one of the EC2 instances.
5. Flask application processes the request.
6. Response is returned to the user.

---

### 3.2.3 Use Case 3: Application Health Monitoring

**Description**

Monitor application health using ALB health checks.

**Actors**

- Application Load Balancer
- EC2 Instances

**Preconditions**

- Health check endpoint configured ( `/` ).
- Flask application running.

**Postconditions**

- Only healthy EC2 instances receive traffic.

**Flow of Events**

1. ALB sends health check request.
2. Flask app responds with HTTP status 200.
3. ALB marks instance as healthy.
4. Traffic is routed to healthy instances only.

---

## 4. Non-Functional Requirements

### 4.1 Performance Requirements

- Application response time should be **less than 2 seconds**.
- Load balancer should distribute traffic evenly across servers.
- Deployment execution via SSM should complete within **2 minutes**.

---

### 4.2 Security Requirements

- EC2 instances should **not allow direct public access**.

- Only **ALB security group** should access EC2 instances.
- SSH access should be restricted to administrators.
- MongoDB Atlas connection string should be stored securely in `.env` file.

---

### 4.3 Usability Requirements

- Deployment should be triggered using a **single SSM Run Command**.
- Logs should be easily accessible at `/var/log/deploy.log`.
- ALB DNS endpoint should allow easy application access.

---

### 4.4 Reliability Requirements

- System should maintain **high availability across multiple AZs**.
- Load balancer should automatically reroute traffic if an instance fails.
- MongoDB Atlas should provide **managed backups and failover**.

---

### 4.5 Compliance Requirements

The system should comply with:

- AWS security best practices
- Data encryption standards
- Access control policies

---

## 5. Assumptions and Dependencies

### 5.1 Assumptions

- AWS account has required permissions.
- EC2 instances have **SSM Agent installed**.
- MongoDB Atlas cluster is already created.
- Flask application repository is available on GitHub.

---

### 5.2 Dependencies

The system depends on:

- AWS EC2 service

- AWS Systems Manager
- AWS Application Load Balancer
- MongoDB Atlas
- GitHub repository availability
- Internet connectivity

---

## 6. Risk Analysis

### 6.1 Risk Identification

| Risk | Description |
|---|---|
| Deployment failure | Code deployment may fail due to dependency issues |
| Instance failure | One EC2 instance may become unavailable |
| Network issues | Connection to MongoDB Atlas may fail |
| Security misconfiguration | Incorrect security groups may expose servers |

---

### 6.2 Risk Mitigation

| Risk | Mitigation |
|---|---|
| Deployment failure | Maintain deployment logs and rollback mechanism |
| Instance failure | Use load balancer across multiple AZs |
| Network issues | Configure retry mechanisms |
| Security issues | Follow AWS security best practices |

---

## 7. Glossary

| Term | Meaning |
|---|---|
| Load Balancer | Distributes network traffic across multiple servers |
| Deployment | Process of updating application code |
| Health Check | Mechanism to verify server availability |
| Cloud Infrastructure | Computing resources hosted in cloud environments |

---

## 8. Appendices

Suggested screenshots to include:

1. EC2 Instances (2 instances running)
2. Application Load Balancer configuration
3. Target Group health check status
4. MongoDB Atlas cluster dashboard
5. SSM Run Command execution
6. `/var/log/deploy.log` output
7. Flask application running via ALB DNS

---

## 9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____